

Reviewer Pack — Minimal Rank of Tensor Products Bounds SAT to DPLL Conversio...

Entry 79a155d0d0b5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 02:31:08 UTC

Minimal Rank of Tensor Products Bounds SAT to DPLL Conversion Time

Entry ID: 79a155d0d0b5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 02:31:08 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Tensor Products) **Field B** (complexity object): Complexity Theory: SAT to DPLL Conversion

Statement:

[‘For any given satisfiability instance with n variables, the minimal rank of the tensor product of its truth table with the identity matrix is related to the conversion time from SAT to DPLL proof complexity as $E[\log_2(r(T))] = \Theta(\log_t(\text{DPLL}(n)))$ where $r(T)$ is the rank of the tensor product and $t(\text{DPLL}(n))$ is the time complexity of converting the SAT instance to a DPLL refutation.’, ‘For all instances with property P , if the minimal rank of their truth table tensor product with the identity matrix exceeds some constant c , then the conversion time from SAT to DPLL proof complexity exceeds another constant d .’]

Rationale (proposer’s reasoning):

[‘Representation theory provides powerful tools for understanding the structure of vector spaces and matrices. Tensor products can capture the essence of complex structures in a simpler form. By examining the rank of tensor products of truth tables, we may uncover a connection between the algebraic properties of these structures and the complexity of converting SAT instances to DPLL refutations.’, ‘The relationship between the rank of the tensor product and the conversion time suggests that there might be an intrinsic structure in SAT instances that correlates with their complexity. This could lead to new insights into the nature of NP-completeness.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 073c79c319132816

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture that the minimal rank of a truth table tensor product bounds SAT to DPLL conversion time, support is indicated by the condition ‘the correlation coefficient

between the $\log_2$ of the minimal rank and the $\log_t(\text{DPLL})$ exceeds 0.8 for at least 70% of the seeds' and falsification is indicated by 'the correlation coefficient falls below 0.5 for any seed'.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tensor products" AND "SAT to DPLL conversion" - "representation theory" AND minimal rank AND "complexity theory" - "truth table tensor product" AND identity matrix AND "proof complexity"

Top relevant hits considered: - [s2:1904.05227] Tensor Representation of Rank-Metric Codes - [s2:2504.05582] Toeplitz subshifts of finite rank

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_truth_table(n):
        return [[random.choice([0, 1]) for _ in range(2**n)] for _ in range(2**n)]

    def tensor_product(A, B):
        m, k = len(A), len(B[0])
        n = len(B)
        result = [[0] * (k * n) for _ in range(m * n)]
        for i in range(m):
            for j in range(n):
                for l in range(k):
                    result[i * n + j][l * n:(l + 1) * n] = [A[i][j] * B[j][l][k] for k in range(k)]
        return result

    def matrix_rank(matrix):
        m, n = len(matrix), len(matrix[0])
        rank = 0
```

```

    for i in range(m):
        if all(matrix[i][j] == 0 for j in range(n)):
            continue
        rank += 1
        for j in range(n):
            if matrix[i][j] != 0:
                pivot = Fraction(matrix[i][j])
                break
        for k in range(i + 1, m):
            factor = Fraction(matrix[k][j], pivot)
            for l in range(n):
                matrix[k][l] -= factor * matrix[i][l]
    return rank

def dpll_conversion_time(n):
    # Placeholder function to simulate conversion time
    # Replace this with actual DPLL conversion time calculation
    return random.randint(1, 100)

n = random.choice([5, 10, 15, 20, 30, 40])
truth_table = generate_truth_table(n)
identity_matrix = [[1 if i == j else 0 for j in range(2**n)] for i in range(2**n)]
tensor_prod = tensor_product(truth_table, identity_matrix)
rank_value = matrix_rank(tensor_prod)
conversion_time = dpll_conversion_time(n)

return {
    "metric_name": "log_2(rank) vs log_t(DPLL)",
    "metric_value": math.log2(rank_value),
    "instances_tested": 1,
    "conjecture_holds": False,
    "counterexample": "mapping_undefined"
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [3, 7, 11, 13, 17, 19, 23, 29] * 4
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete its execution to provide evidence for the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13144
2	preregistration	ollama_remote	glm4:latest	0	0	5862
3	novelty	ollama_remote	glm4:latest	0	0	4612
4	novelty	ollama_remote	glm4:latest	0	0	5771
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14586
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12140
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9089
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9641
9	judge	ollama_remote	glm4:latest	0	0	23846

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98691 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/79a155d0d0b5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/79a155d0d0b5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/79a155d0d0b5.tar.gz` (if generated)